MISA
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

30 Năm PHỤNG SỰ XÃ HỘI 1994 - 2024

Đảm bảo hiệu năng ở tầng Backend

Bùi Ngọc Đức



Phân tải nơi thực hiện công việc

Client (trình duyệt)

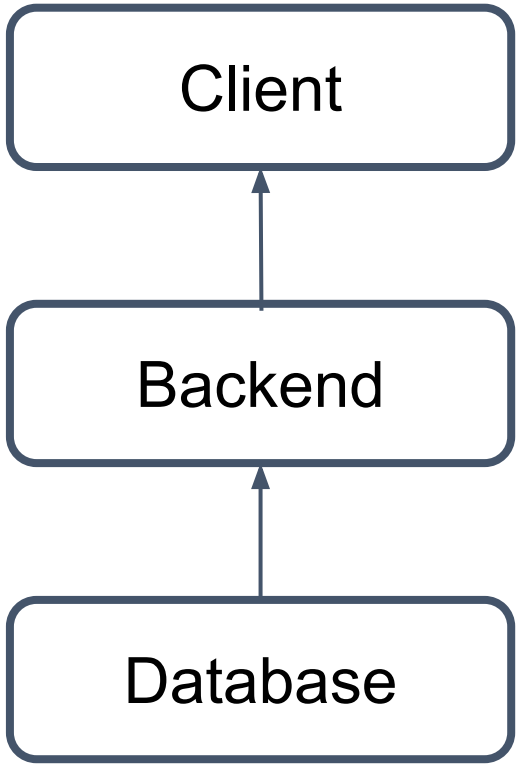
- Sử dụng tài nguyên máy khách hàng
- ***Thực hiện tối đa*** tính toán, xử lý dữ liệu có sẵn ở client để giảm tải cho backend

Backend

- Backend có thể ***scale out dễ dàng hơn database***.
- Thực hiện các công việc không thể làm ở client: Kiểm tra phân quyền, trùng mã, số tồn …
- Không lấy quá nhiều dữ liệu từ database lên Backend để xử lý, Chỉ lấy lên những dữ liệu Backend cần sử dụng

Database

- Database ***khó scale và là nút thắt xử lý***.
- Lưu trữ dữ liệu và thực hiện các truy vấn đọc/ghi dữ liệu



Các vấn đề khi thực hiện tối ưu hiệu năng backend

1. Xử lý cộng/ghép chuỗi
2. Vòng lặp For, ForEach. Vòng lặp lồng nhau
3. Gọi lặp code xử lý
4. Lạm dụng PropertyInfo
5. Async await
6. Sử dụng Cache
7. Sử dụng Queue
8. Sử dụng Using, IDisposable
9. Truy vấn database: Entity framework - Dapper
10. Truyền tải dữ liệu
11. Dependency injection (DI)
12. Clean code
13. Thiết kế lớp

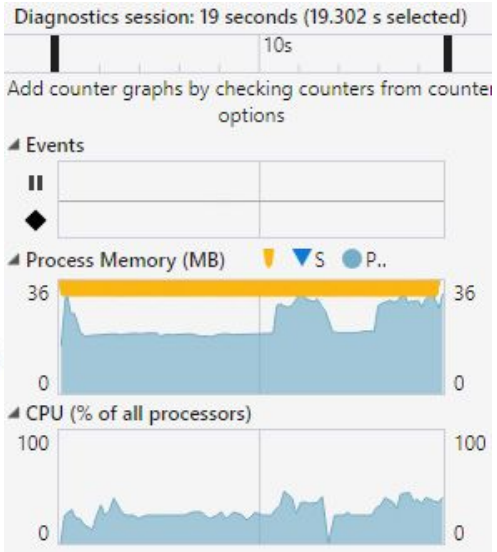
So Sánh: StringBuilder – Cộng chuỗi

Cộng Chuỗi (Không nên làm)

- Tiêu tốn Ram, CPU do tạo nhiều đối tượng chuỗi tạm
- Tốc độ chậm

```
// Cộng Chuỗi
var stopwatch = Stopwatch.StartNew();
var result = "";
for (int i = 0; i < 100000; i++)
{
    result += "test";
}
stopwatch.Stop();
var len = result.Length;
var tm = stopwatch.ElapsedMilliseconds;
Console.WriteLine($"Cộng chuỗi {tm}ms - {len}");
```

```
Cộng chuỗi 19162ms - 400000
```

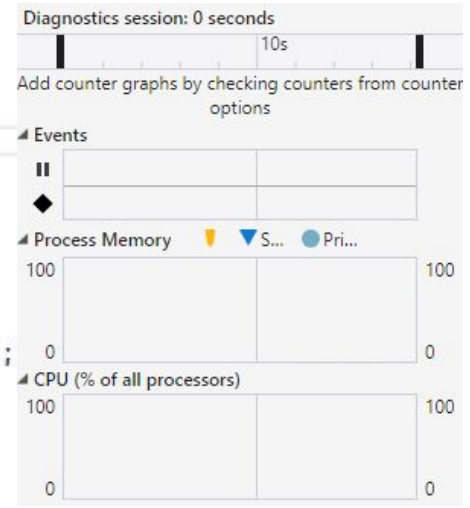


StringBuilder (Nên làm)

- Tiết kiệm tài nguyên do chỉ sử dụng 1 vùng nhớ duy nhất
- Tốc độ nhanh

```
// Sử Dụng StringBuilder
var stopwatch = Stopwatch.StartNew();
var stringBuilder = new StringBuilder();
for (int i = 0; i < 100000; i++)
{
    stringBuilder.Append("test");
}
var result = stringBuilder.ToString();
stopwatch.Stop();
var len = result.Length;
var tm = stopwatch.ElapsedMilliseconds;
Console.WriteLine($"StringBuilder {tm}ms - {len}");
```

```
StringBuilder 2ms - 400000
```



So sánh For – ForEach

- Kiểm nghiệm thì hiệu năng For - ForEach
  - Đo với số liệu ***1 triệu bản ghi mới có khác biệt***
  - Đo với **100k** bản ghi thì đều **= 0** hết
  - Thông thường code nghiệp vụ chỉ duyệt với kích thước mảng < 1000 bản ghi

  → Với code thực tế thì không có khác biệt
- Khuyến nghị **ForEach**: Code ngắn gọn, tiện dụng
- Sử dụng **For(i)** khi:
  - Cần sử dụng vị trí phần tử trong mảng
  - Cần duyệt ngược mảng từ cuối về đầu: **KHÔNG** sắp xếp mảng rồi duyệt xuôi hoặc dùng foreach do tốn hiệu năng sắp xếp

```
public void Test_For_ForEach()
{
    var size = 1000000;
    var list = new List<TestClass>();
    for (var i = 0; i < size; i++)
    {
        list.Add(new TestClass { Value = i });
    }

    var total = 0;
    var sp = new Stopwatch();
    sp.Start();
    for(var i = 0; i <size; i++)
    {
        total += list[i].Value;
    }
    Console.WriteLine($"For:      {sp.ElapsedMilliseconds} ms - total  {total}");
    sp.Stop();

    total = 0;
    sp.Start();
    foreach(var item in list)
    {
        total += item.Value;
    }
    Console.WriteLine($"ForEach: {sp.ElapsedMilliseconds} ms - total  {total}");
    sp.Stop();
}
```

```
For:      9 ms - total  1783293664
ForEach: 18 ms - total  1783293664
```

Vòng lặp lồng nhau

- Code thực tế gặp cần xử lý vòng lặp lồng nhau nhiều
- Số lượng vòng lặp lồng nhau là 1 tiêu chí đánh giá độ phức tạp của hàm: *Càng nhiều càng phức tạp*
- Khi sử dụng các hàm LINQ như *Where, Select, FirstOrDefault*… bản chất cũng là vòng lặp
- Nên duy trì **tối đa 3 vòng lặp lồng nhau** để kiểm soát code tốt và đảm bảo hiệu năng

```
//Lấy ra các dòng chỉ tiêu có công thức chứa chỉ tiêu con truyền vào
//Có công thức khớp với mẫu công thức
reportTemplates.Where(t => !string.IsNullOrWhiteSpace(t.formula) && Regex.IsMatch($"+{t.formula}+", pattem))
    .ToList().ForEach(t =>
    {
        //Dòng chỉ tiêu cha
        var parentItem = datas.FirstOrDefault(i => i.report_item_code?.Equals(t.template_code) ?? false);
        if (parentItem != null)
        {
            //Lấy ra phần tử con trong công thức
            var matchs = Regex.Matches($"+{t.formula}+", pattem);
            foreach (Match match in matchs)
            {
                //Lấy ra dấu trong phép tính
                var sign = Regex.IsMatch(match.Value, pattemSign)
                        ? Regex.Match(match.Value, pattemSign).Value : "+";

                //Duyệt từng cột tiền để xử lý
                //TODO code amountPropertys.ToList() lặp lại nhiều lần
                amountPropertys.ToList().ForEach(p =>
                {
                    //Lấy số tiền cần cộng
                    if (p.GetValue(item) is decimal itemAmount)
                    {
                        //Lấy số tiền chỉ tiêu cha
                        var parentAmount = (p.GetValue(parentItem) is decimal amount) ? amount : 0;
                        switch (sign)
                        {
                            //Phép cộng
                            case "+":
                                parentAmount += itemAmount;
                                break;
                            //Phép trừ
                            case "-":
                                parentAmount -= itemAmount;
                                break;
                            default:
                                break;
                        }
                        p.SetValue(parentItem, parentAmount);
                    }
                });
            }
        }
    });
```

Lạm dụng PropertyInfo

Sử dụng PropertyInfo

- Thực tế review code thấy anh em sử dụng PropertyInfo để đọc ghi dữ liệu **rất nhiều**
- Code rất tiện
- Hiệu năng tệ
- Code không tường minh ngữ cảnh, nghiệp vụ

Sử dụng interface thay thế

- Interface sẽ gắn với nghiệp vụ là gì, sử dụng ra sao → ***thêm thời gian*** thiết kế interface
- Chấm tường minh thuộc tính để đọc ghi dữ liệu → Hiệu năng tốt

```
//code cũ sử dụng getProperyInfo
object data1 = null;
data1.GetType().GetProperty("bank_info_id")?.SetValue(data1, item.bank_info_id);
data1.GetType().GetProperty("bank_info_code")?.SetValue(data1, item.bank_account);
data1.GetType().GetProperty("bank_info_name")?.SetValue(data1, item.bank_name);
data1.GetType().GetProperty("bank_name")?.SetValue(data1, item.bank_name);
```

→

```
//code mới sử dụng interface truy cập trực tiếp thuộc tính
IBankInfo data2 = null;
data2.bank_info_id = item.bank_info_id;
data2.bank_info_code = item.bank_account;
data2.bank_name = item.bank_name;
data2.bank_name = item.bank_name;
```

Gọi lặp code xử lý

- Review code gặp rất nhiều tình huống gọi xử lý lặp lại 1 công việc nhiều lần
- Đặc biệt nghiêm trọng khi đặt trong vòng lặp
  → Giải pháp rất đơn giản: **Tạo biến trung gian** lưu lại để **tái sử dụng.**
  Như ảnh ví dụ tạo 2 biến lưu lại property info của *ref_id* và *ref_type* để tránh gọi lấy liên tục

```
protected override async Task AfterSaveEntity(List<TDtoEdit> dtoEdits, ImportPayload payload)
{
    await base.AfterSaveEntity(dtoEdits, payload);
    //Thực hiện ghi sổ
    using var scope = _serviceScopeFactory.CreateScope();
    var _ledgerService = scope.ServiceProvider.GetRequiredService<ILedgerService>();
    _ledgerService.SetContextData(payload);
    // Ghi sổ những chứng từ thêm mới hoặc sửa
    var errorPosted = await _ledgerService.PostBatchAsync(
        new Application.Contracts.Models.SubmitParameter<List<LedgerDtoPost>>
        {
            IgnoreAll = true,
            Data = dtoEdits.Where(x => x.state == ModelState.Insert || x.state == ModelState.Update)
                           .ToList()
                           .Select(x => new LedgerDtoPost
                           {
                               RefId = (Guid)x.GetType().GetProperty("ref_id").GetValue(x),
                               RefType = (int)x.GetType().GetProperty("ref_type").GetValue(x),
                           }).ToList()
        });
}
```

```
protected override async Task AfterSaveEntity(List<TDtoEdit> dtoEdits, ImportPayload payload)
{
    await base.AfterSaveEntity(dtoEdits, payload);
    //Thực hiện ghi sổ
    using var scope = _serviceScopeFactory.CreateScope();
    var _ledgerService = scope.ServiceProvider.GetRequiredService<ILedgerService>();
    _ledgerService.SetContextData(payload);
    // Ghi sổ những chứng từ thêm mới hoặc sửa
    var type = typeof(TDtoEdit);
    var refId = type.GetProperty("ref_id");
    var refType = type.GetProperty("ref_type");
    var errorPosted = await _ledgerService.PostBatchAsync(
        new Application.Contracts.Models.SubmitParameter<List<LedgerDtoPost>>
        {
            IgnoreAll = true,
            Data = dtoEdits.Where(x => x.state == ModelState.Insert || x.state == ModelState.Update)
                           .ToList()
                           .Select(x => new LedgerDtoPost
                           {
                               RefId = (Guid)refId.GetValue(x),
                               RefType = (int)refType.GetValue(x),
                           }).ToList()
        });
}
```

- Với tình huống kiểm tra Any xong sau đó lấy phần tử đầu tiên để xử lý → Sử dụng luôn FirstOrDefault và kiểm tra có giá trị thì thực hiện để chỉ cần duyệt 1 lần thay vì 2 lần

```
if (props.Any())
{
    //TODO => chỗ này sẽ xử lý trường hợp có 2 cột unique sau
    var propUnique = props.FirstOrDefault();

    _importBaseRepo.SetContextData(payload);
    var values = models.Select(x => propUnique.GetValue(x)).ToList();
    return await _importBaseRepo.GetExists(propUnique, values, typeof(TEntity));

}

//TODO => chỗ này sẽ xử lý trường hợp có 2 cột unique sau
var propUnique = props.FirstOrDefault();
if (propUnique != null)
{
    _importBaseRepo.SetContextData(payload);
    var values = models.Select(x => propUnique.GetValue(x)).ToList();
    return await _importBaseRepo.GetExists(propUnique, values, typeof(TEntity));

}
return new List<object>();
```

Dependency injection (DI)

Hiện trạng

- Khai báo readonly property và truyền qua constructor
- Constructor chưa tuân thủ Single Responsibility principle
- Khởi tạo **tốn nhiều tài nguyên, thời gian**
- Khởi tạo instance xong thường chỉ để **gọi 1 hàm duy nhất,** hàm này sử dụng 1 (1 số) DI được khởi tạo

  Request đến api (Controller) gọi service (BL) đến repo (DL) vào database truy vấn dữ liệu trả cho client.
  - Hàm **GetAsync** chỉ dùng **_employeeRepo,** không dùng **_logger, _syncService**
  - Hàm **SyncAsync** khi có ngoại lệ thì mới cần dùng đến **_logger**

Khởi tạo DI ở constructor **không tốt cho hiệu năng**

```
4 references
public class EmployeeService : IEmployeeService
{
    readonly IEmployeeRepo _employeeRepo;
    readonly ILogger<EmployeeService> _logger;
    readonly ISyncService _syncService;

    0 references
    public EmployeeService(
        ILogger<EmployeeService> logger,
        IEmployeeRepo employeeRepo,
        ISyncService syncService
        )
    {
        _logger = logger;
        _employeeRepo = employeeRepo;
        _syncService = syncService;
    }

    /// <summary> Lấy thông tin nhân viên theo id
    0 references
    public async Task<EmployeeEntity> GetAsync(int id)
    {
        return await _employeeRepo.GetAsync(id);
    }

    /// <summary> Đồng bộ dữ liệu nhân viên
    0 references
    public async Task<bool> SyncAsync()
    {
        try
        {
            return await _syncService.SyncDataAsync("Employee");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"SyncAsync Employee: {ex.Message}");
            return false;
        }
    }
}
```

Dependency injection (DI)

Khuyến nghị

- Constructor chỉ khai báo **IServiceProvider**
- Trong từng hàm khi cần sử dụng DI nào thì sẽ truy xuất qua IServiceProvider
- Tuân thủ quy tắc tối giản tài nguyên, chỉ khởi tạo khi cần sử dụng
- Giảm thời gian, tài nguyên khi khởi tạo đối tượng:

  Đã có sản phẩm lớp BLContact khởi có nhiều DI và DI lồng nhau khởi tạo mất 2s, khi đổi thành chỉ có IServiceProvider thì còn <100ms

  *Nếu sử dụng trong **Task.Run** thì cách này sẽ gây lỗi do web request đã giải phóng, không còn scope để khởi tạo DI → **khởi tạo ngoài Task.run***

```
1 reference
public class EmployeeService2 : IEmployeeService
{
    readonly IServiceProvider _serviceProvider;
    0 references
    public EmployeeService2(
        IServiceProvider serviceProvider
        )
    {
        _serviceProvider = serviceProvider;
    }

    /// <summary> Lấy thông tin nhân viên theo id
    0 references
    public async Task<EmployeeEntity> GetAsync(int id)
    {
        var employeeRepo = _serviceProvider.GetService<IEmployeeRepo>();
        return await employeeRepo.GetAsync(id);
    }

    /// <summary> Đồng bộ dữ liệu nhân viên
    0 references
    public async Task<bool> SyncAsync()
    {
        try
        {
            var syncService = _serviceProvider.GetService<ISyncService>();
            return await syncService.SyncDataAsync("Employee");
        }
        catch (Exception ex)
        {
            var logger = _serviceProvider.GetService<ILogger<EmployeeService>>();
            logger.LogError(ex, $"SyncAsync Employee: {ex.Message}");
            return false;
        }
    }
}
```

Chuyển đổi dữ liệu giữa 2 lớp

- Code thực hiện nhiều chuyển đổi dữ liệu giữa 2 lớp để tính toán dữ liệu, xử lý nghiệp vụ
- Code ví dụ các cách thực hiện cho kết quả như sau:
  - **Clone object** cho tốc độ tốt nhất: code dài, bảo trì ảnh hưởng nhiều khi thay đổi thuộc tính của lớp
  - **AutoMapper, Property info** *chậm hơn* nhưng ở mức chấp nhận được, quan trọng là dễ bảo trì
  - Sử dụng **Serializer/Deserializer** tốc độ rất tệ. *System.Text.Json* gặp lỗi *OutOfMemoryException* với khi kích thước dữ liệu lớn
- Khuyến nghị sử dụng theo thứ tự
  - Clone object
  - AutoMapper
  - Property info

```
Newtonsoft.Json:      192 ms
System.Text.Json:     124 ms
Property Info:         25 ms
AutoMapper:            18 ms
Clone object:           1 ms
```

```
var _source = new List<EmployeeEntity>();
for (var i = 0; i < 10000; i++)...

sw.Restart();
json = Newtonsoft.Json.JsonConvert.SerializeObject(_source);
Newtonsoft.Json.JsonConvert.DeserializeObject<List<EmployeeDtoView>>(json);
sw.Stop(); Console.WriteLine($"Newtonsoft.Json:      {sw.ElapsedMilliseconds} ms");

sw.Restart();
json = System.Text.Json.JsonSerializer.Serialize(_source);
System.Text.Json.JsonSerializer.Deserialize<List<EmployeeDtoView>>(json);
sw.Stop(); Console.WriteLine($"System.Text.Json:     {sw.ElapsedMilliseconds} ms");

sw.Restart();
lstResult = new List<EmployeeDtoView>();
var ps1 = typeof(EmployeeEntity).GetProperties();
var ps2 = typeof(EmployeeDtoView).GetProperties();
foreach (var item in _source)
{
    var tmp = new EmployeeDtoView();
    lstResult.Add(tmp);
    foreach (var pr1 in ps1)
    {
        var pr2 = ps2.FirstOrDefault(n => n.Name == pr1.Name);
        if (pr2 != null)
        {
            pr2.SetValue(tmp, pr1.GetValue(item));
        }
    }
}
sw.Stop(); Console.WriteLine($"Property Info:        {sw.ElapsedMilliseconds} ms");

sw.Restart();
_mapper.Map<List<EmployeeDtoView>>(_source);
sw.Stop(); Console.WriteLine($"AutoMapper:           {sw.ElapsedMilliseconds} ms");

sw.Restart();
lstResult = new List<EmployeeDtoView>();
foreach (var item in _source)
{
    lstResult.Add(new EmployeeDtoView
    {
        Id = item.Id,
        name = item.name,
        dob = item.dob,
        height = item.height,
        inactive = item.inactive,
        unique_key = item.unique_key,
    });
}
sw.Stop(); Console.WriteLine($"Clone object:         {sw.ElapsedMilliseconds} ms");
```

Async await

- Khi gọi xử lý ra ngoài ứng dụng: Gọi httpClient, kết nối/truy vấn database, đọc/ghi/xóa cache redis/couchbase… thông thường sdk luôn cung cấp các hàm đủ 2 overload **sync** và **async**
- Luôn ưu tiên sử dụng các hàm **async** để tận dụng ***Thread Pool*** cho hiệu năng tốt nhất
- Nếu không quan tâm đến kết quả của hàm async nên sử dụng cú pháp **_ =** (fire and forget). *So với **Task.Run** thì hiệu năng khác biệt không đáng kể và đơn giản thi công*
- Chi tiết nội dung này tham khảo [Async/await best practices]

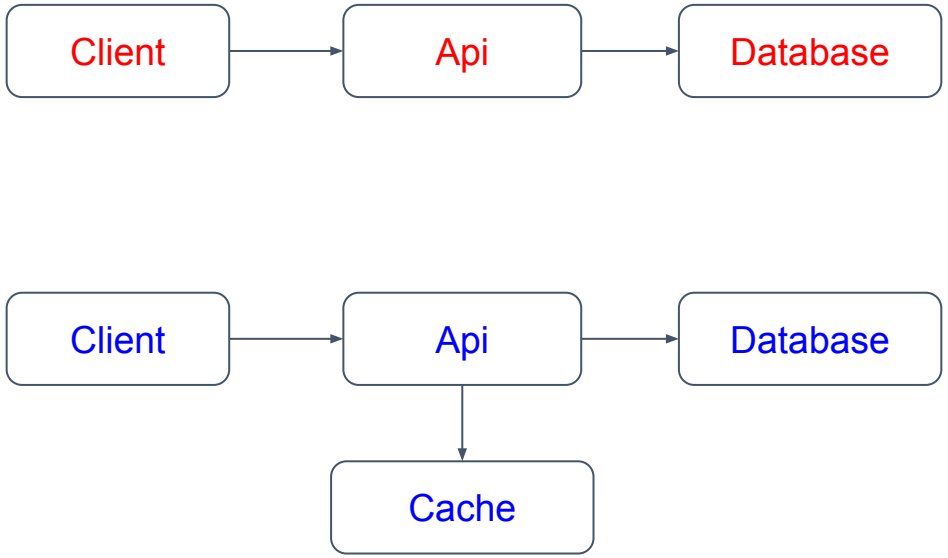
```
Sync :    1300 ms
Async :    890 ms
No await :   4 ms
_ = :        2 ms
Task.Run :   0 ms
```

```
int loopSize = 10;var sw = new Stopwatch();
var httpClient = new HttpClient();
var url = "https://www.google.com/";

sw.Restart();
for (var i = 0; i < loopSize; i++)
{
    var msg = new HttpRequestMessage(HttpMethod.Get, url);
    httpClient.Send(msg);
}
sw.Stop(); Console.WriteLine($"Sync :     {sw.ElapsedMilliseconds} ms");
await Task.Delay(1000);
sw.Restart();
for (var i = 0; i < loopSize; i++)
{
    var msg = new HttpRequestMessage(HttpMethod.Get, url);
    await httpClient.SendAsync(msg);
}
sw.Stop(); Console.WriteLine($"Async :     {sw.ElapsedMilliseconds} ms");
await Task.Delay(1000);
sw.Restart();
for (var i = 0; i < loopSize; i++)
{
    var msg = new HttpRequestMessage(HttpMethod.Get, url);
    httpClient.SendAsync(msg);
}
sw.Stop(); Console.WriteLine($"No await :    {sw.ElapsedMilliseconds} ms");
await Task.Delay(1000);
sw.Restart();
for (var i = 0; i < loopSize; i++)
{
    var msg = new HttpRequestMessage(HttpMethod.Get, url);
    _ = httpClient.SendAsync(msg);
}
sw.Stop(); Console.WriteLine($"_ = :         {sw.ElapsedMilliseconds} ms");
await Task.Delay(1000);
sw.Restart();
for (var i = 0; i < loopSize; i++)
{
    _ = Task.Run(async () =>
    {
        var msg = new HttpRequestMessage(HttpMethod.Get, url);
        await httpClient.SendAsync(msg);
    });
}
sw.Stop(); Console.WriteLine($"Task.Run :    {sw.ElapsedMilliseconds} ms");
```

Sử dụng cache

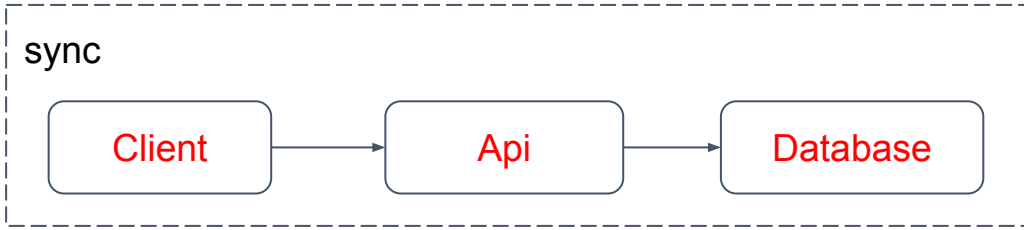
- Thực tế có nhiều dữ liệu **ít thay đổi** thay vì truy vấn đọc lại dữ liệu thì sẽ lưu cache lại để **tái sử dụng**
- Tùy vào **mức độ truy cập**, **dung lượng** dữ liệu sẽ chọn memory cache, distributed cache cho phù hợp
- Khuyến nghị: Dung lượng 1 item lưu cache < **50KB**
- Xử lý invalid cache để tránh dữ liệu trong cache lỗi thời (obsolete) với database
- Chi tiết nội dung này tham khảo
  [Cơ bản về Cache cho MISA SWE]



Sử dụng Queue với nội bộ sản phẩm

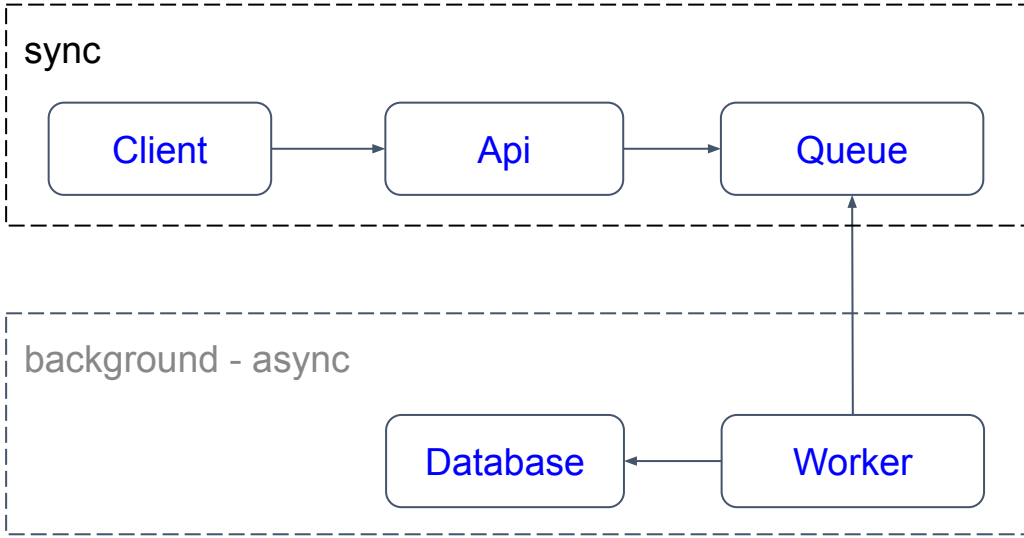
Khi nào cần sử dụng?

- Số lượng request ghi/cập nhật dữ liệu lớn
- Request tốn tài nguyên xử lý: in báo cáo, xuất/nhập khẩu, tính giá, bảo trì…



Lợi ích của việc sử dụng Queue/Worker

- Đảm bảo hiệu năng và ổn định, không bị quá tải khi lượng request tăng đột biến
- Chủ động rollout worker/thread theo dữ liệu tồn trong queue

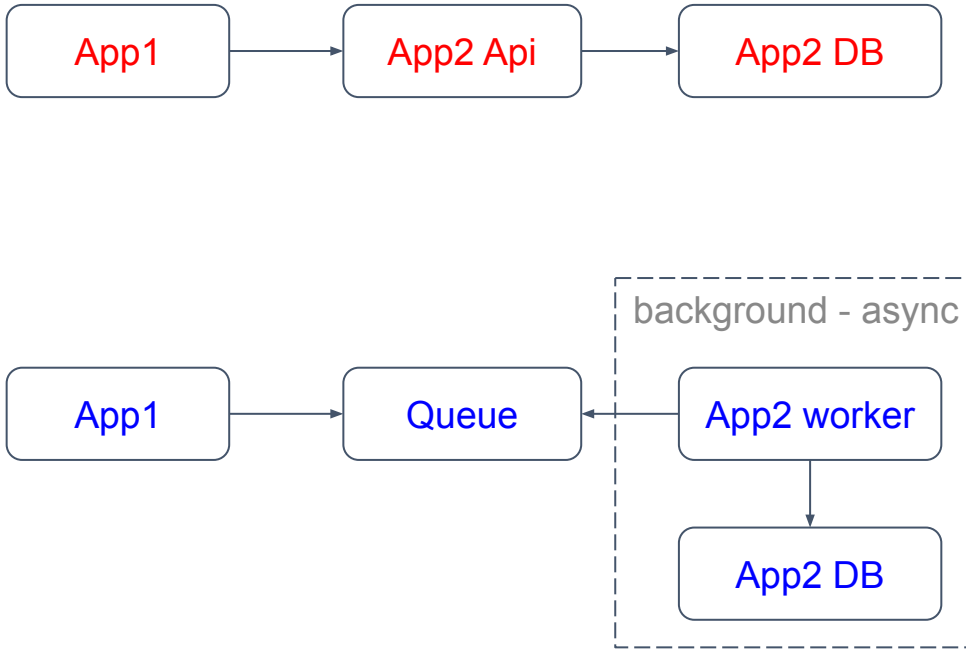


Trải nghiệm người dùng

- Thông báo xử lý bất đồng bộ cho người dùng
- UI vẫn hiển thị loading nhưng thực tế chạy async

Sử dụng Queue tích hợp giữa 2 sản phẩm

- Sử dụng api cho những request kết nối hoặc tương tác trực tiếp cần kết quả ngay
  - Sử dụng api truyền dữ liệu trao đổi giữa thi công nhanh nhưng ảnh hưởng đến hiệu năng ứng dụng
- Sử dụng queue thay cho api khi trao đổi dữ liệu giữa 2 ứng dụng
  - Tính sẵn sàng của hệ thống cao hơn
  - Hiệu năng tốt hơn
  - Dễ dàng mở rộng
  - Khuyến nghị MISA Sync Service



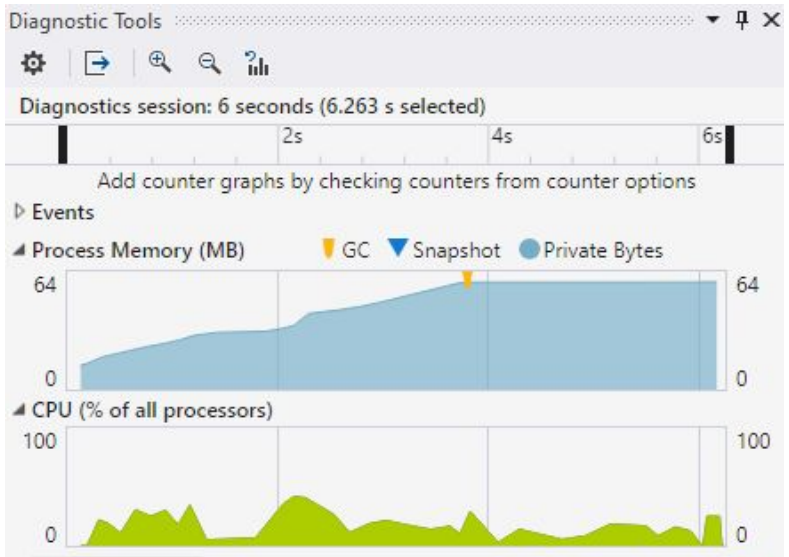
Sử dụng Using, IDisposable

- Khi dev thi công thường ít quan tâm đến việc giải phóng tài nguyên đã khởi tạo sử dụng: Stream, Connection … → rò rỉ bộ nhớ, gây lãng phí tài nguyên của ứng dụng
- Luôn kiểm có hàm Close, Dispose không để gọi giải phóng.
- Khuyến nghị sử dụng **using**
- Nếu có thể thì trước khi kết thúc using gọi **FlushAsync** hoặc sử dụng **await using** để tận dụng thread pool giải phóng tài nguyên
- Khi sử dụng using ***tốn thời gian*** hơn để gọi giải phóng nhưng bù lại ***tiết kiệm*** RAM hơn

```
public async Task TestNotUsing()
{
    var stopwatch = new Stopwatch();
    int size = 10000;
    var bytes = UTF8Encoding.UTF8.GetBytes("xxx");

    // Không sử dụng khối using
    stopwatch.Restart();
    for (int i = 0; i < size; i++)
    {
        var fileStream = new FileStream(
            $"{Guid.NewGuid()}.tmp", FileMode.Create, FileAccess.Write);
        fileStream.Write(bytes, 0, bytes.Length);
    }
    stopwatch.Stop();
    Console.WriteLine($"Not Using: {stopwatch.ElapsedMilliseconds} ms");
}
```

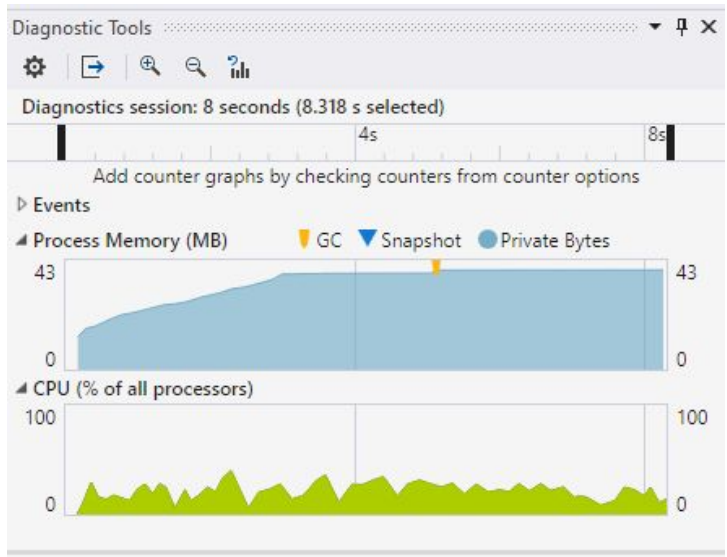
```
Not Using: 2875 ms
```




```
public async Task TestUsing()
{
    var stopwatch = new Stopwatch();
    int size = 10000;
    var bytes = UTF8Encoding.UTF8.GetBytes("xxx");

    // Sử dụng khối using
    stopwatch.Start();
    for (int i = 0; i < size; i++)
    {
        await using (var fileStream = new FileStream(
            $"{Guid.NewGuid()}.tmp", FileMode.Create, FileAccess.Write))
        {
            fileStream.Write(bytes, 0, bytes.Length);
        }
    }
    stopwatch.Stop();
    Console.WriteLine($"Has Using: {stopwatch.ElapsedMilliseconds} ms");
}
```

```
Has Using: 5819 ms
```



Truy vấn database: Entity framework - Dapper

Entity framework

- Với cách coding này rất tiện thi công
- Gặp vấn đề câu lệnh sql sinh ra thường có **cấu trúc phức tạp** (do xử lý tương thích với nhiều tình huống, nhiều loại database) → hiệu năng không tốt
  - Khó tối ưu, điều chỉnh code để câu sql sinh ra đơn giản, có hiệu năng tốt hơn
- Gặp nhiều tình huống GetAll dữ liệu lên rồi lại filter trên code để xử lý dữ liệu → tốn nhiều tài nguyên

Dapper

- Các sản phẩm đều gặp vấn đề hiệu năng với Entity framework → chuyển đổi qua sử dụng **Dapper**, tự build câu lệnh truy vấn dữ liệu để **tối ưu hiệu năng**
- Tự viết câu truy vấn (sql) lấy dữ liệu → chủ động động tinh chỉnh để hiệu năng tốt nhất
- Code sẽ phức tạp hơn do bao gồm cả sql trên file .cs
- 1 số command dài, phức tạp có thể tách ra file text riêng để tiện quản lý hoặc chuyển thành thủ tục trong sql để tối ưu hiệu năng

Truy vấn database: Một số lưu ý khi viết CommandText

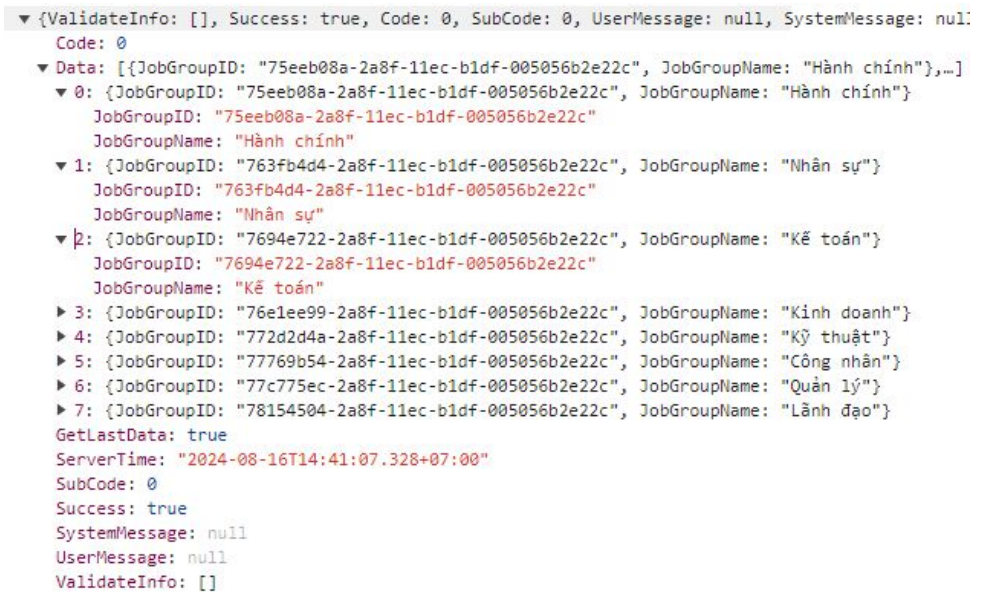
- Không Select * mà cần chỉ ra các cột cần lấy dữ liệu tối ưu truy vấn và dữ liệu truyền tải
  - Lớp *DatabaseProvider, BaseRepo* các hàm có tham số columns chỉ định cột lấy dữ liệu
  - Grid danh sách, combobox: Từ client truyền danh sách cột cần lấy tối ưu truy vấn và truyền tải
- Giảm thiểu số lần gọi không cần thiết vào database
  - Thay vì for gọi Insert từng dòng → InsertBatch
- Với truy vấn lấy từ nhiều bảng, không lấy dữ liệu riêng từng bảng rồi join trên code → Tạo 1 câu truy vấn lấy 1 lần để tận dụng index
  - Có thể sử dụng View
  - Câu truy vấn phức tạp dùng StoreProcedure





Truyền tải dữ liệu

- Khi code xử lý lấy dữ liệu tốt rồi thì thời gian request phụ thuộc vào **khối lượng dữ liệu** truyền tải: Càng nhiều dữ liệu càng tốn thời gian → Chỉ trả kết quả **đúng/đủ** dữ liệu để tối giảm thời gian
  - Đủ số bản ghi (dòng dữ liệu)
  - Đủ số trường thông tin cần thiết
- Ngữ cảnh sử dụng:
  - Api trả dữ liệu cho trình duyệt
  - Dữ liệu distributed cache (redis, couchbase)
  - Dữ liệu trao đổi giữa 2 ứng dụng
- Lưu ý tham số (payload) request chỉ nên truyền đúng/đủ để đảm bảo hiệu năng

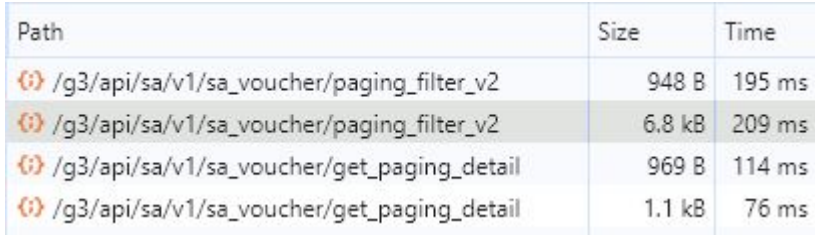


Truyền tải dữ liệu – mẫu thiết kế paging – grid

Grid:

- Tách riêng request lấy dữ liệu, tổng bản ghi
  - Dụng 2 endpoint (url) riêng: ***/data***, ***/summary***
  - Hoặc dùng chung và điều khiển qua tham số
- Request dữ liệu: Lấy từng trang (10, 20… bản ghi), tốc độ nhanh, trả về client hiển thị cho người dùng ngay
- Request summary: Lấy tổng số bản ghi, tính toán tổng dữ liệu các cột liên trang (Số lượng, tổng tiền…). Request này chậm cũng không ảnh hưởng đến trải nghiệm người dùng

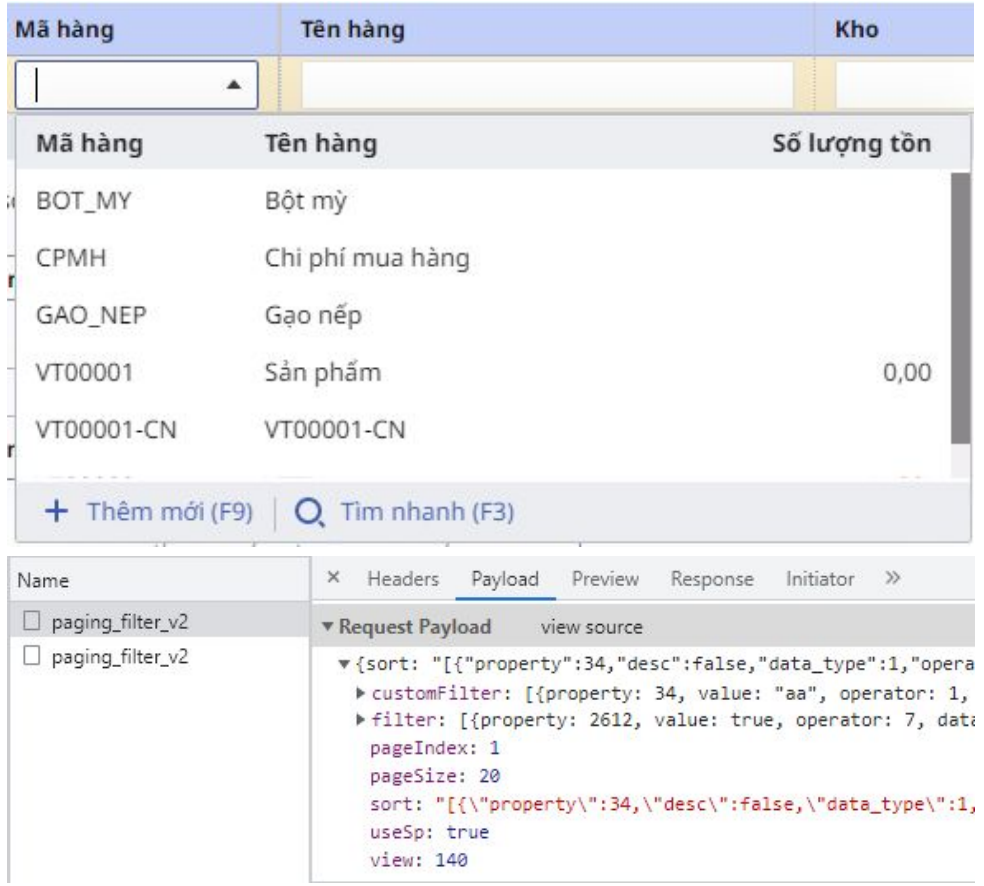




Truyền tải dữ liệu – mẫu thiết kế paging – combobox

Combobox:

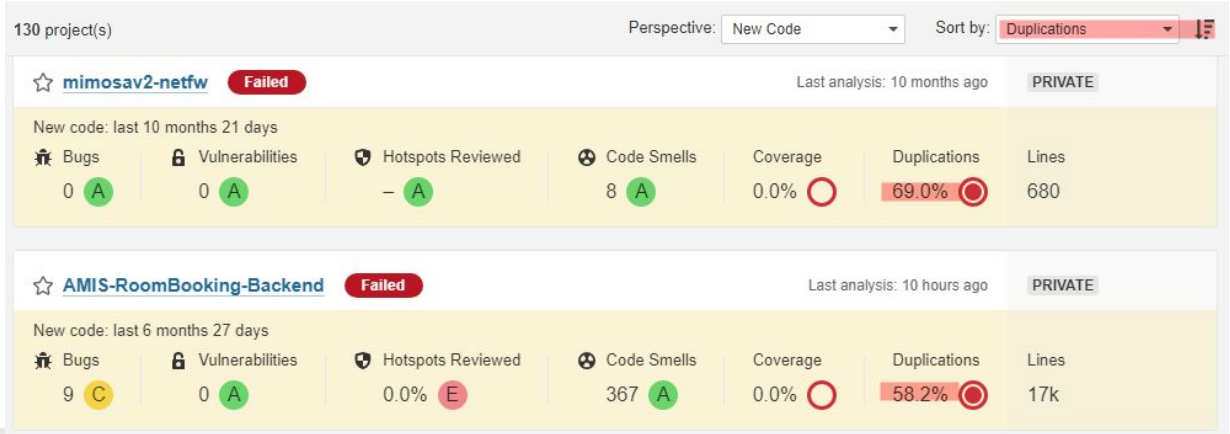
- Với dữ liệu > 150 bản ghi phải load paging
- Xử lý virtual scroll **chỉ gọi request lấy data**, không cần request summary.
  - Cuộn scroll xuống cuối gọi load dữ liệu trang tiếp theo
  - Người dùng thường chỉ cuộn 1 vài lần không tìm thấy sẽ gõ để tìm kiếm
  - Chấp nhận gọi thêm 1 request cuối không có dữ liệu sẽ tốt hơn gọi summary ngay từ đầu.
- Khi mở form không gọi load sẵn dữ liệu của combobox. Khi nào expand hoặc tìm kiếm mới gọi request load dữ liệu



Clean code

Hiện trạng

- Sản phẩm nào cũng có những lớp/hàm kinh điển **dài hàng cây số, code logic phức tạp**: Tính giá, bảo trì, xuất/nhập khẩu…
- Code duyệt vòng lặp, kiểm tra điều kiện lồng nhau rất nhiều → khó đảm bảo hiệu năng
- Chỉ tập trung viết theo case đúng → if lồng nhau rất nhiều




```
605     /// <summary> lấy trước dữ liệu từ danh sách db
        1 reference | 0 changes | 0 authors, 0 changes
610     private void GetImportObjectData(Dictionary<string, ExportImportData> data)
611     {
612         var dataByTable = new Dictionary<string, List<ExportImportData>>();
613         if (data == null || data.Count == 0)...
617         foreach (ExportImportData item in data.Values)
618         {
619             if (!dataByTable.ContainsKey(item.TableName))...
623             else...
627         }
628         foreach (KeyValuePair<string, List<ExportImportData>> item in dataByTable)
629         {
630             //Lấy danh sách db lên
631             IList dataTarget = null;
632             BaseModel model = item.Value[0].Data;
633             string fullNameTable = model.GetExportTableName();
634             string primakeyName = model.GetExportKeyColumnName();
635             Type primakeyType = model.GetExportKeyColumnDataType();
636             ScriptHelperOutput query = model.GetImportObjectDataScript(true);
637             if (!string.IsNullOrEmpty(query.Script))
638             {
639                 if (!_dataInTarget.ContainsKey(item.Key))...
643                 int i = 0;
644                 IList listID = null;
645                 if (primakeyType.Name == typeof(Guid).Name)...
649                 else...
653                 foreach (ExportImportData importData in item.Value)
654                 {
655                     i += 1;
656                     var keyValue = importData.Data.GetExportKeyColumnValue();
657                     if (keyValue != null)...
668                     if (listID.Count >= 1000 || i == item.Value.Count)
669                     {
670                         if (!string.IsNullOrEmpty(query.Script))
671                         {
672                             //chia lô lấy dữ liệu
673                             if (primakeyType.Name == typeof(Guid).Name)...
677                             else...
681                             if (dataTarget != null && dataTarget.Count > 0)
682                             {
683                                 //cập nhật
684                                 foreach (BaseModel itemData in dataTarget)
685                                 {
686                                     object key = itemData.GetExportKeyColumnValue();
687                                     if (!_dataInTarget[item.Key].ContainsKey(key))...
691                                 }
692                             }
693
694                         }
695                         listID.Clear();
696                     }
```

Clean code

Khuyến nghị

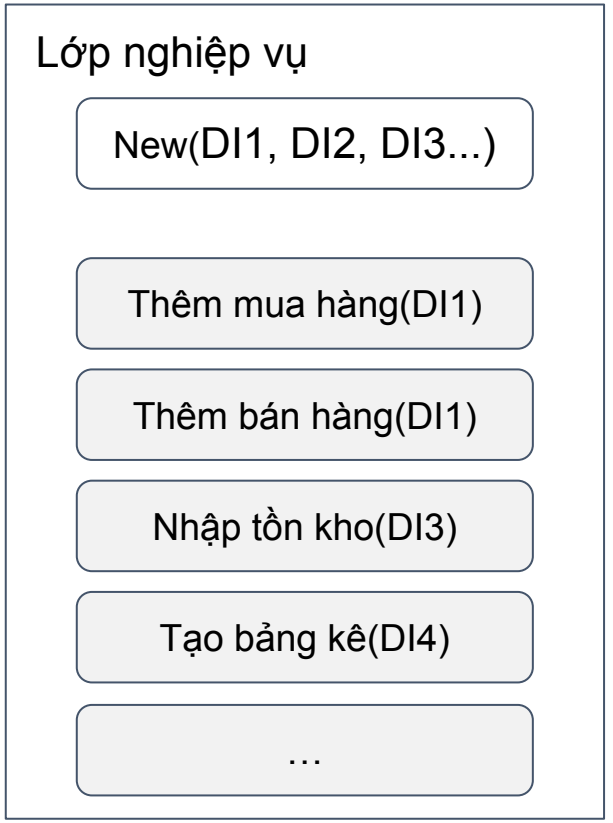
- Viết **điều kiện phủ định** để kết thúc sớm
  - Giảm code block
  - Xử lý đủ case logic fail (case else)
- Nội dung hàm **không dài quá 1 màn hình**. Tách hàm thành nhiều bước, mỗi bước gọi đến 1 hàm thực thi và nhận kết quả xử lý tiếp
- Mỗi hàm chỉ **đủ code thực hiện 1 việc cụ thể**
  - Dễ đọc, tiện bảo trì, giảm rủi ro lỗi effect
  - Tăng tái sử dụng, giảm trùng lặp code
  - Dễ dàng viết unit test kiểm nghiệm code
- Hiện tại công cụ SonarQube đã hỗ trợ thống kê về code block, code trùng lặp … giúp chỉ ra vấn đề kèm hướng dẫn xử lý

```
487     /// <summary> Cất dữ liệu
        3 references | bnduc, 236 days ago | 1 author, 1 change
490     protected virtual async Task<TEntityDtoEdit> SaveAsync(
491         IDbConnection cnn,
492         SaveParameter<TEntityDtoEdit, TEntity> parameter)
493     {
494         //xử lý dữ liệu trước khi cất
495         await this.BeforeSaveAsync(cnn, parameter);
496
497         //validate
498         var validateResult = await this.ValidateSaveAsync(cnn, parameter);
499         if (validateResult != null)
500         {
501             throw new BusinessException()
502             {
503                 ErrorCode = ErrorCodes.Validate,
504                 ErrorData = validateResult
505             };
506         }
507
508         //update data
509         await this.SaveDataAsync(cnn, parameter);
510
511         //Xử lý sau khi cất
512         await this.AfterSaveAsync(cnn, parameter);
513
514         //load bản ghi để đủ thông tin tham chiếu -> hiển thị danh sách
515         if (parameter.ReturnRecord)
516         {
517             var data = await this.GetReturnRecordAsync(cnn, parameter.Model);
518             return data;
519         }
520
521         return default;
522     }
```

Thiết kế lớp: Tham chiếu quá nhiều DI vào lớp

Vấn đề

- Thường đại tiện sử dụng 1 lớp có sẵn để thêm hàm, gắn thêm ***Dependency Injection*** (DI) để thực hiện 1 công việc nào đó không liên quan đến lớp hiện tại
- Sau 1 thời gian lớp rất cồng kềnh và mang tham chiếu đến quá nhiều DI làm cho việc khởi tạo lớp để sử dụng rất phức tạp. Nếu khởi tạo hết DI ở constructor thì **rất tốn tài nguyên** khi khởi tạo đối tượng
- Code nghiệp vụ trong lớp ***chồng chéo,*** nghiệp vụ móc nối ***phức tạp, khó kiểm soát, bảo trì, khó debug***
- Build chạy ứng dụng chậm



Thiết kế lớp: Tham chiếu quá nhiều DI vào lớp

Giải pháp:

- Tuân thủ nguyên lý Solid khi thiết kế lớp
- Tách nhỏ các module nghiệp vụ ra các lớp (interface) riêng, khi cần sử dụng gì thì sẽ kéo vào sử dụng
- Mỗi lớp (interface) chỉ đại diện cho 1 module nghiệp vụ duy nhất
- Giảm độ phức tạp của lớp, dễ kiểm nghiệm, giảm rủi ro ảnh hưởng khi bảo trì
- Tăng tính tái sử dụng, giảm thời gian thi công và giảm code trùng lặp

